

Theory-Cheat-Sheet

OS Theory Cheat Sheet — HIT 61206

What this is

Every **theoretical (concept) question** across all 22 past exams (2014–2024), distilled by section. Compute methods live in the [scheduling](#) and other worked notes. Facts marked  appeared in **2+ exams** → high probability. Facts marked  are common traps. **Disk is off the 2026 exam** (Focus mode) — kept at the bottom for completeness.

1. Systems Concepts

- **Multiprogramming / time-sharing** = several processes kept in memory at once, CPU runs them alternately.
- **Tightly coupled** systems share **main memory (RAM)**; **loosely coupled** have separate memory per CPU.
- **Throughput** = how many processes finish within a fixed time (a performance metric).
- **Benchmarks** are run **to measure computer performance**.
- **Device drivers** (part of the kernel) **mediate between hardware and applications**.
- **Soft real-time** = meets deadlines *most* of the time (not all). (*Hard real-time* = *always*.)
- **SMP (symmetric multiprocessing)** = all CPUs run the **same kernel**, no "master/slave" — vs **asymmetric** (one master CPU controls the rest). Linux & Windows are symmetric.

2. Interrupts & Mode

- **Two CPU modes:** I/O instructions & device access run only in **kernel mode**; in kernel mode you can read/write **all of RAM**.
- **ISR (Interrupt Service Routine)** = kernel function that responds to a hardware event / interrupt.  (asked ~3×)
- **Interrupt vector** = table holding the **address** of the function the CPU jumps to on an interrupt.
- **Trap = a software-generated interrupt** (software interrupt).  "What makes a trap special? → *it's software-generated*."
- **System-call instruction** does two things: switches CPU to **kernel mode** and jumps to **kernel code**.
- **Timer interrupt** = the hardware interrupt raised when a process's **quantum expires**.

- **Windows nested interrupts:** if a new interrupt arrives mid-handling, its ISR runs **after** the current one finishes (sequential model — that's the course answer).

3. Processes & States

- **Ready** = wants to run, waiting for a free CPU.
- **Context switch** = stop one process on the CPU and run another. Registers are saved in the PCB.
- **PCB (Process Control Block)**  = kernel structure holding a process's **page-table address, nice value, owner, registers**, etc. (everything about a non-running process).
- **Linux state letters**  (very commonly tested):
 - **R** = running or runnable — including **busy-waiting** (still R!).
 - **S** = interruptible sleep — waiting on I/O, or writing to a **full pipe buffer**, or on a device request queue.
- **Zombie / orphan:**
 - **wait()** (called by parent) frees a **zombie** child's PCB.
 - A zombie's PCB is *also* released if **the parent dies**.
 - **0** processes can be zombie AND orphan at once (init reaps orphans). 
- **Exit status** = the success/failure number a child sends its parent on termination (set via the **exit()** syscall).
- **Threads of one task share** the **code + data (+ heap)** segments; each thread keeps its own **stack & registers**.
- **User-level threads** = scheduled by **library functions**; **kernel-level (supported) threads** = scheduled by **system calls**.
- **wait() with no children** → returns **-1**, process just **keeps running**. 
- **execv()** replaces the process image, so code written **after a successful execv()** **never runs** — it runs **only if execv() failed**. 

4. Scheduling → full notes: [Scheduling/Scheduling-Theory](#)

- **CPU burst** = executing **without I/O** between two I/O requests (e.g. "add two registers"). 
- **Response time** = start → **first output**. **Waiting time** = total time in the **ready queue**.  (*don't confuse the two*)
- **Preemptive scheduler** = can move a process **running** → **ready**. Examples: **SRTF**, **Round Robin**.
- **RR preempts when the quantum (time slice) expires**. **RR(quantum = ∞) = FCFS**. 
- **Starvation** = low-priority processes never run (priority scheduling) → fix = **aging**.
- **Schedulers by horizon:** **long-term** (job admission — on batch / supercomputers, absent on desktop PCs), **short-term** (CPU dispatch), **medium-term** (swap in/out).

- **Non-preemptive** classes: an infinite **CPU-only loop (no I/O)** can freeze the machine — the CPU can't be taken by force. ⚠️
- **FCFS** → **convoy effect**: short jobs pile up waiting behind one long job.
- **Windows**: focus-window process gets a **3× longer quantum**. 🔄 (asked 2×)
- **Linux**: one round over all CPU-requesting processes = an **epoch**; scheduling approach = **Multilevel Queue** (*course answer*; *real Linux* = CFS).

5. Synchronization 🔄 (biggest section — 53 Qs)

Critical-section conditions (memorize all three — each asked separately):

- **Mutual Exclusion** = only one process in the CS while writing shared memory.
- **Progress** = guarantees **no deadlock** (someone can always proceed).
- **Bounded Waiting** = bounds how many others enter before a waiting process gets in. ⚠️
Its violation → **starvation** (e.g. readers-writers starves writers).
- "*Starvation*" is a **consequence**, not a condition. ⚠️

Bakery algorithm:

- Software-only CS solution (no OS calls), works for **any number** of processes.
- Compares **PIDs only when ticket numbers are equal** → **smaller PID** enters first.

Busy waiting & race conditions:

- A CS solution that **doesn't call the OS** can't deschedule → causes **busy waiting**.
- **Race condition** = concurrent access to a shared resource gives **order-dependent results**.

Semaphores (concept):

- On **wait** making the value **negative** → kernel **adds the process to the semaphore's wait queue** (it sleeps).
- On **signal** when value not positive → kernel **dequeues a waiting process to ready**.
- A negative value's magnitude = **number of processes waiting**.
- **Deadlock** needs a minimum of **2 binary semaphores**. Dining philosophers (each holds left, waits for right) = **deadlock**.
- The OS can't use semaphores to schedule across cores because **the kernel can't go to sleep**.
- **Init value**: **1** = mutex (one process in the CS); **0** = **ordering/signalling** (force F of A before G of B — B waits, A signals); **N** = allow N at once.
- **Indirect communication** = the general name for IPC where processes pass data through a **kernel-allocated region** instead of writing each other's memory.
- **Binary** ↔ **counting semaphores** are **inter-implementable** — each can be built from the other.

- **In-kernel critical sections** use **TestAndSet** on a shared boolean (the kernel can't sleep, so it spins). ⚠

Bounded-buffer (producer-consumer):

- `empty` positive = **number of empty slots**; `full` negative = **number of consumers waiting**.

Readers-writers: writer inside → **0 readers** allowed. An exiting reader signals a writer **when no readers remain**.

Shared memory: shared region size must be a **multiple of the page size**.

Linux IPC syscalls ⚠ (memorize the verbs):

- **semget** = allocate a semaphore array (its 2nd arg = array size; `semget(IPC_PRIVATE, 123, ...)` → 123 semaphores).
- **semctl** = **delete** semaphores (`IPC_RMID`) **and initialize** their values.
- **semop** = wait/signal ops; on a **deleted** semaphore returns **-1**; `sem_op=-1` on a sem valued **0** → process **sleeps/blocks**.
- One **semop()** with an N-op array is **atomic** (all-or-none) → atomic wait-5 + signal-5 = **1 call** (nsops=10).
- Setting `sem_op` but **never calling semop()** → **no operation happens**.
- **shmget** allocates / **shmctl(IPC_RMID)** frees shared memory; **shmdt** on a bad pointer → **-1 / EINVAL**.
- Shell: `ipcrm -s 666` deletes semaphore set 666.
- Semaphores **not removed explicitly** (`semctl IPC_RMID`) persist until the machine **crashes / reboots**.

6. Memory & Paging

- **MMU** = hardware that **translates virtual** → **physical** addresses. Done in hardware (not OS) because it's **CPU-independent** and runs on **every memory access** (far faster than trapping).
- **TLB (Translation Lookaside Buffer)** = **cache of the page table**.  Avoids the extra RAM access a page-table lookup costs; the key speedup for **multilevel paging**.
- **Valid-invalid bit** off → MMU **traps to OS** = **page fault**. 
- **Page fault** brings the needed page from **disk** → **RAM**; the faulting instruction is **re-executed** (return to *the instruction that faulted*).
- **Demand paging:** a new process's page table starts **empty**; a page loads **only when its data is requested** → only **pages actually used** are in RAM.
- **Internal fragmentation** = waste from allocating in whole-**page** multiples. Writing into that slack in Linux → **no fault, value just written**.

- **PTBR** (page-table base register): **1** per CPU core; reloaded on **every context switch** (each process has its own page table).
- **Copy-on-Write** = child shares parent's pages until a **write** triggers a copy.
- **Two-level split (Linux)**: low level = **pages**, high level = **segments**.
- **Dirty / modify bit** = set by the **MMU (hardware)** when a page is written → lets the OS **skip writing back unmodified** pages on eviction.
- **shmat** = **attach** a shared-memory segment into the address space (pairs with **shmdt** = detach); dereferencing the pointer **after shmdt** → **segmentation fault**.
- **Swap partition / swap space** = the disk area that stores **pages evicted from RAM**.

7. Page Replacement

- **Thrashing**  (asked 3×) = processes need **more frames than available** → OS spends most time **swapping** instead of running.
- **Optimal (OPT)** evicts the page **used farthest in the future**.
- **LFU (Least-Frequently-Used)** evicts the least-used page; kept in a **heap** → **O(log n)** per memory access.
- **FIFO** suffers **Belady's anomaly** (more frames → more faults). 
- **Clock algorithm**:
 - Victim search = **O(N)**. 
 - The **OS writes 0** to the reference bit (clears it); on a frame with reference bit **set**, Clock **clears it** and moves on.  (*CPU/MMU sets the bit; OS clears it.*)
- **Local allocation**: to bring a page for process X, evict **a page of X itself**.

8. Disk & Storage **OFF the 2026 exam (kept for reference)**

- **Spool** = dedicated **hard-disk** area buffering output for a **slow device** so apps don't block (e.g. mail/print). Sits on the **HDD**.
- Devices with **no spool**: **fast** output devices — **screen/monitor, keyboard**.
- **Caching** = move data from **slow storage** → **faster storage**.
- **DMA** = hardware that copies disk data **into RAM** without the CPU; most of an I/O's time is the transfer into the **disk controller's buffer** → RAM.
- **Printer daemon** = the process that moves a file from the **spool** to the printer.
- **FAT / linked allocation** = a file is stored as a **linked list of blocks**.
- **Mount** = attach one disk's directory tree **under a directory** of another disk's tree.
- **Cylinder** = the set of tracks at the same arm position across all platters — readable **without moving the arm** (*disk geometry — off the 2026 exam*).
- **Cheapest medium** (exam answer) = **HDD**. (*Strictly, magnetic tape is cheapest — but write HDD.*)

★ Highest-probability facts (appeared 2+ times)

- **ISR** = kernel's interrupt handler · **PCB** holds everything about a non-running process
- **Linux R vs S** (busy-wait = R!) · **TLB** = page-table cache, saves a RAM access
- **Valid-invalid bit off** → **page fault** · **Thrashing** = over-committed frames, all swapping
- **RR($q=\infty$)=FCFS** · **Windows focus window = 3× quantum** · **CPU burst** = compute between I/O
- **3 CS conditions:** Mutual Exclusion / Progress / Bounded Waiting (starvation = consequence)